TEST_STRATEGY

Status: Draft v1. Candidates: Vitest (unit and integration), Playwright (end to end), pytest (Python). CI is out of scope for now, so all checks run locally with npm scripts.

1. Goals

Prove the rules that protect users: ownership, approval binding, state machine, no code execution, no push to main. Tests must show evidence, not only pass counts.

2. Test layers

Layer	Purpose	Tool	What it covers
Unit	Small pure logic	Vitest (backend, frontend), pytest (aiServices)	State transitions, approval validation, path safety, stack detection, diff and manifest match, schema validators, error codes
Integration	Modules together with real MongoDB and Redis (local test instances) and a mocked GitHub	Vitest	API routes, conditional updates, idempotency, races, cross-user access, queue jobs, worker steps
Security and negative	Hostile inputs	Vitest with fixture repos	Malicious hooks, symlinks, traversal paths, huge repos, prompt injection text, forged repo names, stale approvals
AI contract	AI output handling	Vitest and pytest	Malformed or malicious AI output is rejected. Fake AI provider is used by default.

Layer	Purpose	Tool	What it covers
End to end	Full user flow in a real browser	Playwright	Login (mocked OAuth), repo select, findings, plan, approval, review, push permission
Manual and exploratory	Real GitHub and real AI	You	Run a real flow on a throwaway test repo before any release

3. Test data (fixtures)

- clean-js-app: normal React app.
- python-app: requirements and pyproject.
- monorepo-a (frontend and backend folders) and monorepo-b (apps and packages): same detection result expected.
- malicious-hooks: git hooks and package scripts that write a marker file. Marker must never appear.
- symlink-repo: symlinks to files outside the snapshot.
- traversal-repo: file names with ../ patterns.
- huge-repo: exceeds size and file count limits.
- injection-repo: comments saying ignore rules and modify other files.
- broken-syntax-repo: syntax errors to test static checks.

Fake GitHub server and fake AI provider are used so tests need no network and no cost.

4. Critical test list

ID	Test	Expected
TS-1	Select repo owned by another user	REPO_NOT_OWNED, no workflow
TS-2	Select private repo, org repo	Rejected
TS-3	Banned user calls any route	403
TS-4	Cross-user workflow access	404

ID	Test	Expected
TS-5	Clone malicious-hooks	No marker file, no process spawned
TS-6	Symlink and traversal fixtures	No file read outside snapshot
TS-7	huge-repo	SNAPSHOT_TOO_LARGE, temp files cleaned
TS-8	Folder rename of monorepo	Same detected profile
TS-9	Findings without evidence	Labelled uncertain
TS-10	injection-repo	Plan scope not widened, state not changed by AI
TS-11	AI service down	Rule-based findings shown, flow continues
TS-12	Approve twice (parallel)	One approval
TS-13	Change SHA then approve or execute	APPROVAL_STALE
TS-14	Reuse approval for other plan	Rejected
TS-15	Execute with out-of-scope op	OP_OUT_OF_SCOPE
TS-16	Crash worker during EXECUTING, retry	Same V2, no duplicate files
TS-17	Diff files versus manifest	Exact match, unapproved change flagged
TS-18	Broken syntax in changed file	Flagged in static checks
TS-19	Open PR without permission	PUSH_PERMISSION_MISSING, no PR created
TS-20	Open PR twice (parallel)	One PR, no duplicate commits
TS-21	Target branch main	BRANCH_PROTECTED. No force-push or merge function exists.

ID	Test	Expected
TS-22	Branch name exists	Suffix -2 added
TS-23	Before approval, the only GitHub write is creating the work branch. No write ever targets main (mock counter)	0
TS-24	Token and key scan on logs and responses	None found
TS-25	Data delete with active job	Job cancelled, files and records gone, verified
TS-26	Rate limit, daily cap, AI limit	429 with proper code
TS-27	Repo content with script tags in UI	Shown as text, not executed
TS-28	Review page text	Contains warning that code was not run

5. Quality gates before merging own changes

Lint (for Obsidian's own code), unit and integration tests, security test list, dependency audit (npm audit, pip audit), secret scan, frontend build. These run locally for now. When you decide to add CI, the same commands go into GitHub Actions.

6. Rules

- A failing check stops work. Investigate before changing more code.
- Skipped tests and warnings are reported, never hidden.
- Each phase ends with a test report you review.
- Passing tests never means production-ready. Release criteria are in DEFINITION_OF_DONE.

7. Open items

Coverage targets. Choice of fake GitHub approach. Whether to use an in-memory MongoDB or a local instance for tests.